

Osiris: A Systolic Approach to Accelerating Fully Homomorphic Encryption

AUSTIN EBEL, Electrical and Computer Engineering, New York University, New York, United States

BRANDON REAGEN, Electrical and Computer Engineering, New York University, New York, United States

In this article, we demonstrate how fully homomorphic encryption (FHE) can be accelerated using a systolic-inspired architecture. We start by analyzing FHE algorithms and then design dedicated systolic or systolic-esque units for each major kernel. Connecting these units is challenging due to opposing data access patterns. We overcome this by developing a new data tiling approach, named limb interleaving. Here, data from independent residues (limbs) are fed to the units in a round-robin fashion rather than processing each limb to completion before the next. This choice creates a common data input-output pattern across kernels that allows the entire architecture, named Osiris, to operate in lockstep. The Osiris accelerator is systolic in nature. It comprises a set of simple processing cells (homogeneous within each unit), runs in lockstep, and omits many of the costly structures found in other architectures (e.g., register files, NoCs, and chaining). It is capable of processing key-switches, bootstrapping, and full neural network inferences with high utilization across a range of FHE parameters. To achieve high performance, we propose a new dataflow that efficiently maps state-of-the-art FHE matrix-vector products onto Osiris to promote reuse and parallelism. Our evaluation confirms that Osiris achieves performance on par with or exceeding previous state-of-the-art accelerators across all standard benchmarks.

CCS Concepts: • **Computer systems organization** → **Parallel architectures**; • **Security and privacy** → **Cryptography**;

Additional Key Words and Phrases: Fully homomorphic encryption, systolic arrays, hardware acceleration, hardware/software co-design

ACM Reference Format:

Austin Ebel and Brandon Reagen. 2026. Osiris: A Systolic Approach to Accelerating Fully Homomorphic Encryption. *ACM Trans. Arch. Code Optim.* 23, 1, Article 21 (March 2026), 27 pages. <https://doi.org/10.1145/3788287>

1 Introduction

The ubiquity of cloud computing and online services has caused wide-scale data outsourcing, raising privacy and security concerns. This has sparked an increased interest in techniques for

This work was supported in part by **Graduate Assistance in Areas of National Need (GAANN)**. The research was developed with funding from the NSF CAREER award #2340137 and from DARPA, under the **Data Protection in Virtual Environments (DPRIVE)** program, contract HR0011-21-9-0003. Reagen and Ebel received generous support from the NY State **Center for Advanced Technology in Telecommunications (CATT)** and a gift award from Google.

Authors' Contact Information: Austin Ebel (corresponding author), Electrical and Computer Engineering, New York University, New York, New York, United States; e-mail: abe5240@nyu.edu; Brandon Reagen, Electrical and Computer Engineering, New York University, New York, New York, United States; e-mail: bjr5@nyu.edu.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 1544-3973/2026/03-ART21

<https://doi.org/10.1145/3788287>

privacy-preserving computation and confidential computing, including **fully homomorphic encryption (FHE)** and **zero-knowledge proofs (ZKPs)** [19–21]. These techniques enable evaluating functions directly on encrypted data or proving correctness without revealing inputs, ensuring data is always secured. One popular technique for cryptographic confidential computing is FHE. FHE provides secure outsourced computation using emerging encryption schemes, making it a natural fit for today’s cloud model. While promising, the broad deployment of FHE remains limited due to the significant performance overhead it introduces.

Prior work has made tremendous strides in addressing these overheads and spans multiple domains. Cryptographers have devised new FHE schemes and optimizations for better performance (e.g., SIMD ciphertext packing [70], hoisting [9, 35], and CKKS’s native fixed-point support [15]). Compiler research has assuaged many of FHE’s programming complexities while simultaneously improving performance, including instruction scheduling [18, 54, 74], SIMD ciphertext packing and data layout [5, 26, 44], and bootstrap placement [8, 16, 26]. Finally, work on hardware acceleration has proposed architectures that provide the computational power needed to overcome the slowdown. Early solutions targeted fixed-function designs to demonstrate the magnitudes of slowdown could be overcome [63, 64]. Others have developed vector architectures, a natural fit as the core data structure in FHE (polynomials) readily maps to the ISAs [30, 65, 71]. Recent efforts have included tiled/clustered architectures and FPGAs. In tiled solutions, processing lanes and NoCs can be reformed to execute different FHE kernels [46, 47, 50]. FPGA-specific designs provide immediate speedup running on existing commercially available hardware [2, 3, 64, 69]. The insights and speedups of prior work are substantial, yet room for improvement remains. For example, fixed-function ASICs lack the flexibility to support a variety of FHE workloads nor algorithms; vectors are flexible but rely on complex circuitry for generality and performance (e.g., register files and chaining); tiled/clustered may be difficult to program and require large NoCs.

In this article, we develop a systolic architecture to accelerate FHE, named *Osiris*. Systolic architectures address the problems noted above. Rigid, lock-step communication eliminates the need for data sharing via register files (or chaining), NoC wiring, and minimizes control overhead. At the same time, flexibility in data layout and staging provides the necessary programmability to support a range of FHE parameters and workloads. The main intellectual insight of this article is recognizing and distilling the complexity of FHE into a set of relatively simple computational cells with rigid communication patterns and devising a dataflow that enables composing them into a complete systolic architecture.

Osiris makes multiple contributions to accomplish this goal. First, we propose and develop individual systolic units for each FHE kernel (except for the automorphism, which we show can be mapped to a Beneš network), see Section 4 for details. When necessary, we also consider how kernels interact and co-design units with those they connect with to optimize interfaces. Effectively integrating units into a complete systolic architecture is non-trivial. Kernels such as the **number theoretic transform (NTT)** naturally operate on individual limbs, whereas **basis conversions (BConv)** operate *across* limbs. Since these kernels happen in succession, naïve solutions would require large buffers between the units to reorganize data, leading to pipeline stalls. Our second contribution, *limb interleaving*, solves this issue. Limb interleaving forces data from separate limbs to be accessed and operated on in a round-robin fashion. In doing so, we eliminate the large inter-unit buffers and stalls present in prior accelerators. As each unit is systolic, the I/O rates are straightforward to balance. With limb interleaving and the developed systolic units, we compose a complete systolic architecture capable of accelerating entire FHE workloads, including bootstrapping and neural network inferences.

Efficient hardware is only part of the challenge. *Osiris* provides high-performance kernel accelerators and efficient interfaces between them, leaving flexibility in how FHE is mapped to it.

Table 1. CKKS Parameter Notation and Descriptions

Param.	Description
$\mathbf{m}$	Message, a vector of real or complex numbers.
$[P]$	Plaintext polynomial encoding the message, $\mathbf{m}$.
$[[C]]$	Ciphertext encrypting the plaintext polynomial, $[P]$.
N	Power-of-two polynomial ring degree.
n	Length (slots) of the vector message, $n \leq \frac{N}{2}$ (or N).
L	Maximum multiplicative level of $[[C]]$.
ℓ	Current multiplicative level.
Q	Initial polynomial modulus.
q_i	Small moduli in RNS decomposition of $Q = \prod_{i=0}^L q_i$.
α	Limbs in hybrid decomposition basis of Q .
dnum	Decomposition number in key-switching, $\lceil \frac{(\ell+1)}{\alpha} \rceil$.
P	Auxiliary modulus used in key-switching.
p_i	Small moduli in RNS decomposition of $P = \prod_{i=0}^{\alpha-1} p_i$.
L_{boot}	Number of levels consumed by bootstrapping.
L_{eff}	Maximum achievable level after bootstrapping.

Optimizing FHE dataflow is complex and compounded by recent algorithmic advancements to computing matrix-vector products, specifically the **baby-step giant-step (BSGS)** algorithm [35] and double-hoisted rotations [9]. Double-hoisting is a recent optimization that substantially reduces the absolute number of modular multiplications within homomorphic matrix-vector products by cleverly reusing intermediate results. This creates two challenges: masking data movement latency with fewer computations and deciding which data to cache on-chip. Our third contribution is the development of the **giant-step centric (GSC)** dataflow, which addresses both challenges. Osiris supports optimizations that reduce bandwidth, i.e., **on-the-fly limb (OF-Limb)** extension [46, 47], key generation [65], and twiddle generation [49]. In GSC, we schedule kernels such that OF-Limb overlaps with loading evaluation keys from off-chip, and we directly size Osiris to balance these two latencies.

This article makes the following contributions:

- (1) We develop unique systolic or systolic-esque units for each FHE kernel that seamlessly compose to form a larger systolic architecture, named Osiris.
- (2) We develop limb interleaving and show that it eliminates the inter-unit buffers used in prior FHE accelerators, removing a major source of pipeline stalls.
- (3) We propose the GSC dataflow to map efficient FHE matrix-vector products onto Osiris.
- (4) We rigorously evaluate Osiris under standard benchmarks and with sensitivity studies to demonstrate design balance and high utilization. Osiris achieves state-of-the-art bootstrap performance and a 6417 $\times$ speedup over CPU, and we show near linear performance improvements with increased resources.

2 Background

In this section, we introduce the relevant parameters, notation, and operations of CKKS [15]. An overview of the notation is shown in Table 1. When possible, we adopt terminology from [2, 46, 50]. CKKS supports several homomorphic operations on and between plaintext polynomials and ciphertexts, similar to prior HE schemes such as BGV [10] and BFV [11]. Specifically, we denote the additions (multiplications) of two ciphertexts as HAdd (HMult) and the rotation of a ciphertext

ALGORITHM 1: $\text{BConv}_{\alpha \rightarrow \beta}([B_{\text{coeff}}])$

Input: $[B_{\text{coeff}}]$, an input polynomial of size $\alpha \times N$, and bt , a base table matrix of scaling factors of size $\beta \times \alpha$.

Output: The polynomial with β limbs and modulus Q' also in the coefficient representation.

```

1 for  $i = 0; i < \beta; i = i + 1$  do
2   for  $j = 0; j < N; j = j + 1$  do
3      $psum \leftarrow 0$ 
4     for  $k = 0; k < \alpha; k = k + 1$  do
5        $x_{out} \leftarrow \text{SwitchModulus}(B_{\text{coeff}}[k, j], q[k], q'[i])$ 
6        $psum \leftarrow (psum + bt[i, j] \cdot x_{out}) \pmod{q'[i]}$ 
7      $C_{\text{coeff}}[i, j] \leftarrow psum$ 
8 return  $[C_{\text{coeff}}]$ 

9 Function  $\text{SwitchModulus}(x_{in}, q, q')$ :
10    $\Delta \leftarrow (q - q') \pmod{q'}$ 
11   if  $\Delta > \lfloor q'/2 \rfloor$  then
12      $x_{in} \leftarrow (x_{in} + \Delta)$ 
13   return  $x_{in}$ 
14 end

```

by HRot. To perform these operations, a vector message, $\mathbf{m}$, is first encoded into a polynomial $[P] = \mathbb{Z}_Q[X]/(X^N + 1)$, with coefficients modulo Q , and ring degree N . A ciphertext, $[[C]]$ is a pair of polynomials that can be generated from $[P]$, and is made secure by adding a small amount of random noise to one of the polynomials.

Since Q is often large (on the order of thousands of bits), it is most efficient to decompose a polynomial into many separate residues, or *limbs*, each with a smaller size modulus, q_i , such that $Q = \prod_{i=0}^L q_i$, using the Chinese remainder theorem [14]. L sets the ciphertext's maximum *level*, and operations such as HMult consume levels. When the current multiplicative level is zero, $\ell = 0$, an expensive Bootstrap procedure is required to increase the ciphertext's level to enable further computation. Bootstrapping consumes a fixed number of levels, L_{boot} , and therefore a ciphertext can only reach an *effective* level, $L_{\text{eff}} = L - L_{\text{boot}}$ after bootstrapping.

Both HMult and HRot operations transform a ciphertext originally decrypted by secret key, s , into a ciphertext only decryptable by a new secret key, s' . The most efficient approach to return to s and enable further computation is to multiply the ciphertext with a special *switching* key, swk , to perform a re-encryption of the ciphertext under the old secret key, s . However, a naïve multiplication with the swk adds prohibitively large noise. Techniques proposed by [28] and later by [37] address this issue directly, with a common theme being that they first Decompose a ciphertext into many *digits* and perform this KeyMult operation at a higher modulus. The act of changing a ciphertext's modulus is expensive, requiring many **(inverse) number theoretic transforms (I/NTTs)** and BConv operations. In Osiris, we collectively refer to changing the RNS representation of a modulus from α limbs to β limbs as ModChange. When $\alpha < \beta$ ($\alpha > \beta$), we refer to the operation as ModUp (ModDown). The Rescale operation is a specific case of ModDown when $\beta = \alpha - 1$.

Algorithm 2 describes this ModChange operation. Note that separate I/NTTs must be applied to each limb, $[A]_{q_i}$, and that the modulus changes from $Q = \prod_{i=0}^{\alpha-1} q_i$ to $Q' = \prod_{j=0}^{\beta-1} q'_j$. We also emphasize that the core operation in BConv is a matrix-matrix multiplication between a *base table* of scaling factors and the input polynomial. In Section 4, we show how to accelerate this operation with a 2D systolic array.

ALGORITHM 2: $\text{ModChange}_{\alpha \rightarrow \beta}([A])$

Input: $[A]$, a polynomial of α limbs with modulus Q , and s , preliminary limb-wise scaling factors of BConv .

Output: The polynomial with β limbs and modulus Q' .

- 1 $[A_{\text{coeff}}] \leftarrow \text{INTT}([A]_{q_i})_{0 \leq i < \alpha}$
- 2 $[B_{\text{coeff}}] \leftarrow ([A_{\text{coeff}}] \cdot s_i) \pmod{q_i}$
- 3 $[C_{\text{coeff}}] \leftarrow \text{BConv}_{\alpha \rightarrow \beta}([B_{\text{coeff}}])$
- 4 **return** $\text{NTT}([C_{\text{coeff}}]_{q'_j})_{0 \leq j < \beta}$

3 FHE Matrix-Vector Products

We now introduce how fast homomorphic matrix-vector products are implemented. We start with the naïve approach and then describe two compounding optimizations: the BSGS algorithm and *hoisting*. To the best of our knowledge, Osiris is the first ASIC design to adopt state-of-the-art hoisting techniques [9, 35] for matrix-vector products. These computations can account for 70% to 99% of the private inference latency in FHE [26], and therefore, much of Osiris’s architecture and mapping strategies are tailored to efficiently perform matrix-vector products.

Diagonal Method: The simplest approach to FHE matrix-vector products is the diagonal method [34]. Here, one first extracts the *generalized diagonals* of a matrix, $\mathbf{M}$. That is, diagonals of the form $\text{diag}_k = \mathbf{M}_{[0,k]}, \mathbf{M}_{[1,1+k]}, \dots, \mathbf{M}_{[w-1,w-1+k]}$, where w is the width of the matrix and the second index is taken modulo w . Each diagonal is multiplied by a rotated input ciphertext. diag_k begins with the element in column k of $\mathbf{M}$, resulting in rotating the input by k slots in order to align the plaintext diagonal and the ciphertext. Figure 1(a) shows the approach, where the number of non-zero diagonals, n , is 6. Note that since we have rotated the ciphertext n times, we must perform n distinct key-switch operations, each computing a Decompose, KeyMult, and ModDown, to realign partial products under the same secret key. Thus, the runtime can be seen as $\mathcal{O}(n)$.

BSGS: The BSGS algorithm reduces the runtime complexity of homomorphic matrix-vector products to $\mathcal{O}(\sqrt{n})$ [35]. Therefore, as the number of non-zero diagonals increase, so does the impact of this optimization. We show the BSGS approach in Figure 1(b) and modify the running matrix-vector example from Figure 1(a). Here, matrix-vector products are parameterized by two constants: n_1 , the number of baby steps or input rotations, and n_2 the number of giant steps or partial product rotations, where $n_1 n_2 = n$. In Figure 1(b), we set $n_1 = 3$ and $n_2 = 2$ such that $n_1 n_2 = 6$. The distinction between n_1 and n_2 lies in the magnitude of their rotations and when the rotations are performed. We first generate the n_1 input rotations (baby steps) of the input ciphertext. Each baby-step rotation is then multiplied with n_2 diagonals.

Unlike in the diagonal method, we are left with n_2 partial products, each offset by some multiple of n_1 . Therefore, n_2 partial rotations (giant steps) are required to realign ciphertexts before the final reduction. Rotations are expensive, and the key insight of BSGS is that we can choose to reuse ciphertext rotations and *instead* rotate matrix diagonals (Line 7, Algorithm 3). Since matrix diagonals are known *a priori* (weights in convolutions or a constant matrix to bootstrapping), they can be trivially rotated before being encoded. This observation reduces the number of ciphertext rotations to just $n_1 + n_2$, and rotations are minimized when $n_1 = n_2 = \sqrt{n}$.

Hoisting: Hoisting is a cryptographic optimization that further reduces the complexity of matrix-vector products. This technique can be applied to stages of the BSGS algorithm that share a common result. As we will see, rotations to the *same* input ciphertext share a common Decompose output. Therefore, we can perform this computation only once, *hoisting* it out and reusing its result for all

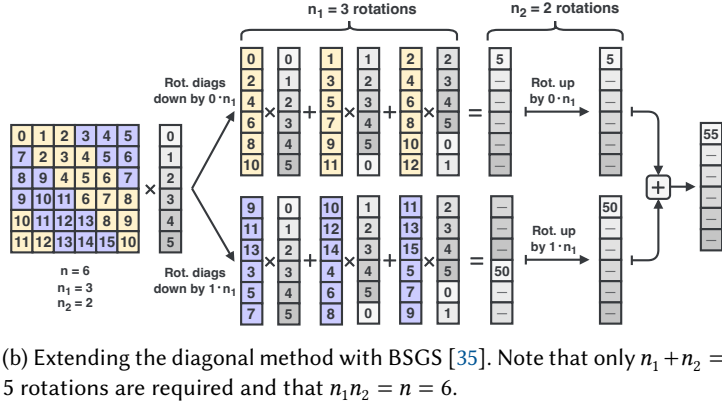
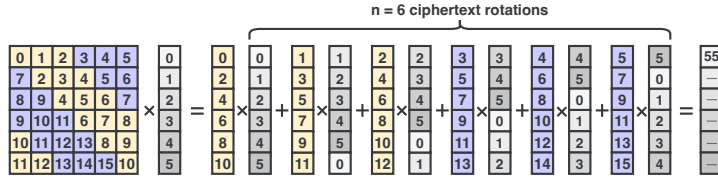


Fig. 1. Visualizing how the BSGS algorithm [34] reduces the number of ciphertext rotations in matrix-vector products.

ALGORITHM 3: BSGS matrix $\times$ vector [35]

Input: ct encrypting vector message, $\mathbf{m}$, $\mathbf{M}_{\text{diag}}$ the diagonals of $\mathbf{M}$, an $n \times n$ matrix with $n = n_1 n_2$.

Output: $\text{ct}' = \mathbf{M} \times \text{ct}$.

```

1 for  $i = 1; i < n_1; i = i + 1$  do
2    $\text{ct}_i \leftarrow \text{HRot}_i(\text{ct})$ 
3  $\text{ct}' \leftarrow (0, 0)$ 
4 for  $j = 0$  to  $n_2; j = j + 1$  do
5    $\mathbf{r} \leftarrow (0, 0)$ 
6   for  $i = 0, i < n_1; i = i + 1$  do
7      $\mathbf{r} \leftarrow \text{HAdd}(\mathbf{r}, \text{PMult}(\text{ct}_i, \text{Rot}_{-n_1 \cdot j}(\mathbf{M}_{\text{diag}}^{n_1 \cdot j + i})))$ 
8    $\text{ct}' \leftarrow \text{HAdd}(\text{ct}', \text{HRot}_{n_1 \cdot j}(\mathbf{r}))$ 
9 return  $\text{ct}'$ 

```

n_1 baby-step rotations. We further discuss the state-of-the-art double hoisting algorithm [9] and its mapping onto Osiris in Section 5.

Convolutions as Matrix-Vector Products: We note one final optimization specific to private neural network inference. On Osiris, we convert convolutions to their equivalent matrix-vector form to benefit from hoisting in all network layers. For this conversion, we adopt the single-shot multiplexing method from Orion [26]. We refer interested readers to the article for a thorough explanation of data packing strategies that support hoisting.

4 The Kernel Accelerators

In this section, we introduce the hardware components that comprise Osiris. We begin by establishing a consistent notation for polynomials to help contrast our design with prior work. Then,

Limb 3	0 ₁₂	1 ₁₃	2 ₁₄	3 ₁₅	4 ₁₂	5 ₁₃	6 ₁₄	7 ₁₅	8 ₁₂	9 ₁₃	10 ₁₄	11 ₁₅	12 ₁₂	13 ₁₃	14 ₁₄	15 ₁₅
Limb 2	0 ₈	1 ₉	2 ₁₀	3 ₁₁	4 ₈	5 ₉	6 ₁₀	7 ₁₁	8 ₈	9 ₉	10 ₁₀	11 ₁₁	12 ₈	13 ₉	14 ₁₀	15 ₁₁
Limb 1	0 ₄	1 ₅	2 ₆	3 ₇	4 ₄	5 ₅	6 ₆	7 ₇	8 ₄	9 ₅	10 ₆	11 ₇	12 ₄	13 ₅	14 ₆	15 ₇
Limb 0	0 ₀	1 ₁	2 ₂	3 ₃	4 ₀	5 ₁	6 ₂	7 ₃	8 ₀	9 ₁	10 ₂	11 ₃	12 ₀	13 ₁	14 ₂	15 ₃

limb coefficient index_{input/output order}

Fig. 2. Matrix representation of a polynomial ($N = 16$, $\ell = 3$) in natural order. Element i_o indicates that the coefficient at index i within its limb is in the o 'th set of coefficients to be processed by a hardware component. The highlighted element 0_0 shows that the first coefficient of the first limb is in the first set of four coefficients to be processed.

we discuss the benefits of pipelined FFT architectures in FHE and their implementation in Osiris. Next, we show how the BConv operation can be accelerated with a single 2D systolic array. Finally, we discuss how Osiris performs automorphisms and pointwise arithmetic. In the next section, we show how to seamlessly connect each of these components to perform the matrix-vector products introduced in Section 3.

4.1 A Running Example

Consider the small running example of a polynomial with ring degree $N = 16$ and multiplicative level $\ell = 3$ (four limbs). This polynomial can be represented as a 4×16 matrix with each element denoted by i_o . Figure 2 illustrates this matrix representation. The index i represents the position of the coefficient within its limb (not the coefficient itself). In Figure 2, the polynomial is shown in *natural* order, with indices monotonically increasing from left to right. The same polynomial can also be represented in *bit-reversed* order, which can be distinguished by visualizing only its reordered indices. The subscript, o , indicates the order that coefficients enter or leave a hardware component. For example, the element highlighted in red in Figure 2 conveys that the first coefficient of the first limb is in the initial set of four values to enter or leave a component.

As we introduce each of Osiris's hardware components in the following subsections, we will refer back to this example to concretize the concepts. The order that coefficients enter and leave functional units in Osiris is different from prior FHE accelerators. Our key insight is that interleaving the computations of separate limbs enables the lockstep execution of larger FHE kernels such as ModChange. Therefore, explicitly writing each coefficient in a polynomial as i_o allows us to better compare this new dataflow with prior work.

4.2 Accelerating I/NTT

The Multi-Delay Commutator (MDC): The MDC belongs to a family of pipelined architectures introduced in the 1970s to accelerate the **Fast Fourier Transform (FFT)** [31, 32]. In Osiris, we employ a radix-2 pipelined MDC architecture to accelerate the I/NTT. Two parameters define each MDC unit: s , the number of stages in the design, setting its depth, and p , the number of parallel inputs and outputs to each stage, defining its width.

Radix-2 pipelined architectures perform the Cooley–Tukey FFT algorithm [17]. This algorithm decomposes an N -point DFT into $\log_2(N)$ stages, each containing $\frac{N}{2}$ butterfly operations. By design, each stage in the pipelined architecture performs *all* the butterfly operations corresponding to the same stage in the Cooley–Tukey dataflow. Therefore, any radix-2 design will have $s = \log_2(N)$ stages. A p -parallel MDC design consists of $\frac{p}{2}$ butterfly *cells* per stage. Each cell accepts two inputs and performs one butterfly operation per cycle. In the Cooley–Tukey dataflow, the width of each butterfly (i.e., the difference in coefficient indices between inputs) varies from stage to stage. The MDC achieves this same variability through a series of fixed buffers and multiplexers placed between each stage to *commute* the order of inputs. In total, each MDC has $N - p$ buffers. We refer

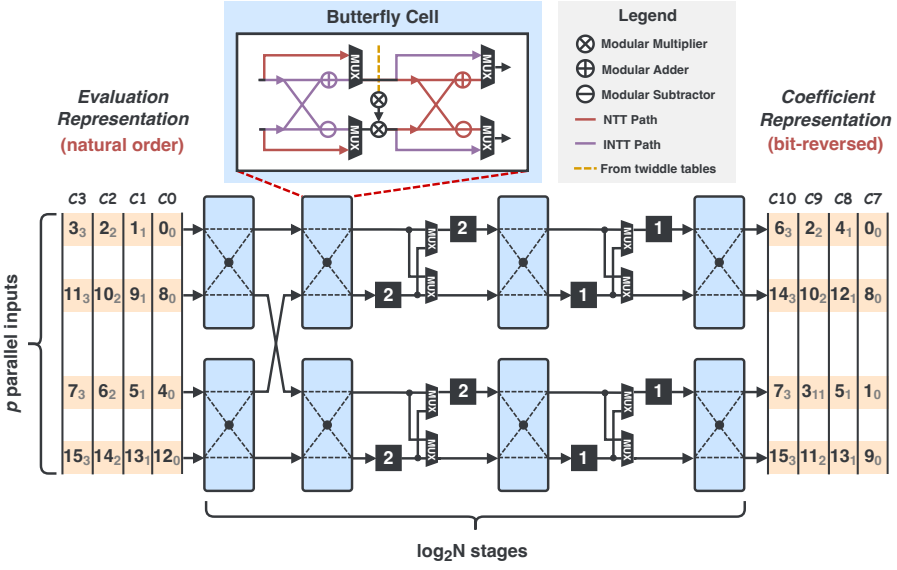


Fig. 3. A smaller 4-parallel MDC architecture performing an INTT on the first limb of the example polynomial introduced in Figure 2. The *same* input ordering shown above is compactly described by the i_0 notation in Figure 2. C0 denotes cycle 0. Buffer lengths are shown between stages.

interested readers to [32] for a more thorough explanation of the history and theory of pipelined FFT architectures.

Adapting for Osiris: Osiris supports a maximum ring degree of $N = 2^{16}$. Therefore, we set $s = 16^\dagger$. We set the width of each MDC to be $p = 512$ after carefully evaluating the bandwidth characteristics of BSGS hoisting (see Section 5). Figure 3 visualizes a small 4-parallel MDC architecture performing INTT on the first limb of the example polynomial introduced in Figure 2 and highlights three key aspects of the design. First, each butterfly cell contains *two* modular multipliers, one for **on-the-fly twiddle (OF-Twiddle)** factor generation [49] and another for the butterfly operation itself. OF-Twiddle decomposes N unique twiddle factors into two tables of size $\sqrt{N}$, enabling the generation of any twiddle factor by multiplying two entries, one from each table. This decomposition reduces the storage requirement of twiddle factors from 13 MB to just 0.20 MB, with groups of $p = 16$ lanes sharing a common set of twiddle tables. Second, pipelined FFT architectures are bi-directional: data flowing in one direction computes the FFT, while the other direction performs the inverse FFT. Figure 3 illustrates this concept, with the highlighted butterfly cell featuring both INTT and NTT paths. For clarity, we omit drawing the additional control logic between stages that routes both directions [53]. In Section 5, we show how the bi-directional nature of pipelined NTT architectures can double the throughput of the DiagMult kernel. Third, MDC architectures always accept and produce p elements per cycle. In the following subsection, we detail how this constant flow of data simplifies rate-matching with our BConv hardware.

Interleaving I/NTTs: Beyond their aforementioned benefits, we choose pipelined NTT architectures for their ability to easily *interleave* the computations of independent I/NTTs. Recall from Algorithm 2 that I/NTT operations occur in the ModChange kernel as part of the $\text{INTT} \rightarrow \text{BConv} \rightarrow \text{NTT}$ routine. Each I/NTT acts on unique *rows* (limbs) of the matrix represen-

[†]For any $N' < 2^{16}$, we skip the first $16 - \log_2(N')$ stages in the design.

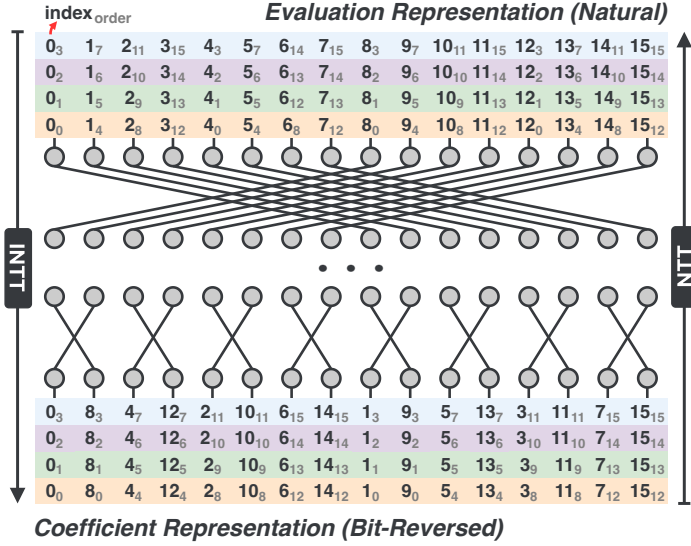


Fig. 4. Visualizing the interleaved input and output order of the 4-parallel MDC unit introduced in Figure 3 using i_o notation introduced in Section 4.1.

tation introduced in Section 4.1. By contrast, recall that the BConv operation is fundamentally a matrix multiplication. Rows of a base table are multiplied with *columns* of an input polynomial. Each column in this polynomial contains coefficients from *all* its limbs. Therefore, performing INTTs on each limb sequentially stalls the ModChange pipeline until all limbs have been processed.

In Osiris, we avoid these stalls by forcing the output order of the MDC INTT to match the column-wise input order required by BConv. Figure 4 visualizes this new *interleaved* I/NTT format using the i_o notation from Section 4.1, and the smaller 4-parallel MDC introduced above. Note that while the INTT input indices are identical to those in Figure 2, the subscripts are different. Specifically, here we begin by feeding $p = 4$ values from the first limb to our MDC INTT unit. In the next cycle, rather than the next four values from the first limb, we instead feed the first four values from the second limb, beginning a second INTT. We continue this interleaving process for all limbs in the polynomial before returning to the first limb for its second set of four values. As a result, coefficients flow from the MDC INTT in the column-wise order required by BConv, allowing the two units to operate together in lockstep.

Differences from Prior Work: Figure 5 captures the differences in INTT output (BConv input) ordering between Osiris and prior FHE accelerators. Note that the BConv operation in Osiris can begin well before any single INTT completes. This change to dataflow is achieved simply by extending the buffers between each MDC stage by a factor equal to the number of interleaved limbs, compensating for the increased delay between butterflies of the same I/NTT. Therefore, the 512-parallel MDC units on Osiris have 13 MB of buffers.

Design Summary: I/NTT is implemented with a radix-2 MDC architecture. It has $s = \log_2 N$ stages ($s \leq 16$) and processes $p = 512$ elements per cycle. Consecutive cycles interleave p elements from distinct limbs in a round-robin fashion to align with the BConv’s column-wise input order and enable lockstep execution.

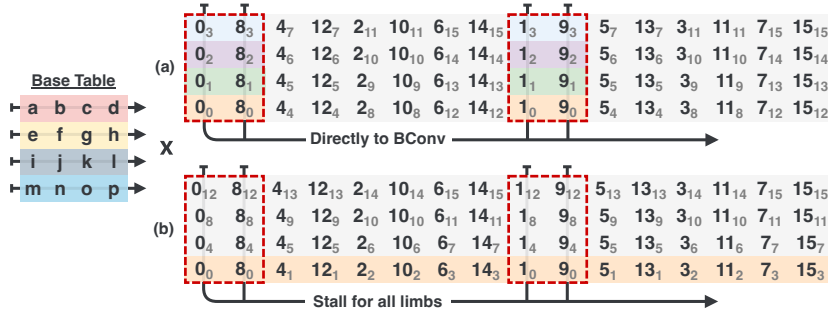


Fig. 5. BConv input ordering in (a) Osiris and (b) prior FHE accelerators using the same example polynomial introduced in Section 4.1. Highlighted elements indicate coefficients available after four cycles. Note that in Osiris, we can begin the BConv operation immediately since *rows* of a base table are multiplied with *columns* of the input polynomial.

4.3 Accelerating BConv

2D Systolic Arrays: Given that the core operation of BConv is a large matrix-matrix multiplication, it is natural to consider accelerating this kernel with a 2D systolic array. Systolic arrays, like pipelined FFT architectures, gained prominence in the 1970s by offering large degrees of parallelism through grids of locally connected, homogeneous **processing elements (PEs)**. Today, systolic arrays remain one of the most commercially viable tools for accelerating large matrix multiplications and currently power much of Google’s deep learning workloads in the TPU [41, 42].

In a systolic array, inputs flow through the device in a pipelined fashion. For matrix multiplication, each PE performs one **multiply-accumulate (MAC)** operation per cycle, contributing to a partial sum that is also passed between neighboring PEs. Outputs stream from PEs at the periphery of the array. When the array’s dimensions are smaller than the dimensions of the input matrices, blocked matrix multiplication algorithms are required.

Adapting for Osiris: In Osiris, we accelerate BConv with one large 2D input-stationary systolic array. The height of our systolic array is determined by the largest height (or number of limbs) of any input polynomial passed to BConv. In hybrid key-switching [37], all BConv operations act on input polynomials of at most α limbs; either as the size of each decomposed digit in ModUp or as the number of limbs in the auxiliary modulus, P , in ModDown (see Table 1). We set the maximum supported $\alpha = 16$ in order to minimize the size of key-switching keys while remaining 128-bit secure. Therefore, we also size the height of the systolic array to be 16 cells tall. Likewise, we size the BConv array to be 512 cells wide to seamlessly accept the MDC’s output of $p = 512$ coefficients per cycle. As a result, the MDC naturally partitions the larger $\alpha \times N$ input polynomial into smaller blocks, each of size $\alpha \times p$, that are then fed directly to the systolic array.

Figure 6 illustrates a smaller 4×4 BConv systolic array processing the interleaved polynomial introduced in Figure 4, with $N = 16$ and $\ell = 3$. For consistency, we use the same base table as in Figure 5, converting a input polynomial from modulus $Q = \prod_{i \leq 3} q_i$ to a new modulus $Q' = \prod_{i \leq 3} q'_i$. We highlight four features in Figure 6. First, interleaved coefficients (*top*) and base table constants (*left*) are first skewed by buffers of differing lengths before entering the array. Skewed input matrices are common to all systolic architectures and ensure that the correct partial sums (*psums*) are accumulated within the array. Second, input polynomial coefficients are double-buffered. We reserve two registers within each PE for input coefficients: one for the coefficient from the current $\alpha \times p$ (4×4 in Figure 6) block being processed, and another for preloading coefficients from the next block. Third, all PEs perform the same **switch-modulus multiply-accumulate (SMAC)**

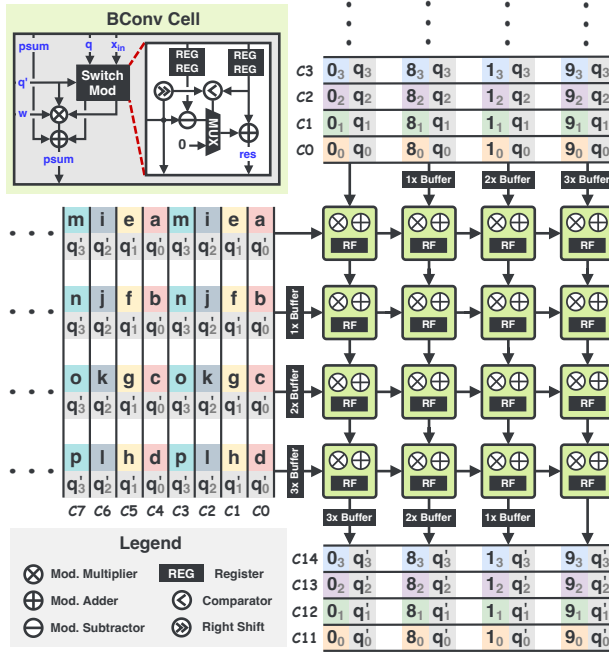


Fig. 6. A 4×4 BConv array converting $\ell = 3$ interleaved input limbs (mod Q) into $\ell = 3$ interleaved output limbs (mod Q'). The first row of PEs each contain an additional modular multiplier for the preliminary limb-wise scaling in BConv. For clarity, only the psum output is shown.

operation. For a base table constant w , an input coefficient x_{in} , and an old (new) modulus q (q'), we define this operation as:

$$\text{psum} \leftarrow \text{psum} + w \cdot \text{SwitchModulus}_{q \rightarrow q'}(x_{in}) \pmod{q'}$$

Since the SwitchModulus function involves both the old modulus, q , and the new modulus, q' , we stream each q'_i into the array alongside its corresponding row of base table constants. This ensures that all partial sums are accumulated under the same new modulus. Finally, output coefficients leave the array in an interleaved fashion and are unskewed by a set of complementary buffers. Since BConv is part of the larger ModChange pipeline, keeping coefficients interleaved ensures that they can be passed directly to subsequent MDC NTT hardware. Hereafter, we assume that all operations, including element-wise operations and automorphisms, act on polynomials in an interleaved manner.

Differences from Prior Work: Notably, ARK [47] and SHARP [46] employ systolic arrays to accelerate BConv. Since these designs do not interleave limbs, problematic memory access patterns force the authors to allocate 1024×6 (ARK) or 2×8 (SHARP) output-stationary systolic arrays, one per lane. Therefore, unique base table constants must be broadcasted across the 256 lanes in each cluster using central **broadcast units (BrUs)**. In contrast, the interleaved ordering in Osiris enables base table constants to be passed systolically between PEs in one large 2D array.

Design Summary: BConv is implemented with one $\alpha \times 512$ input-stationary systolic array ($\alpha \leq 16$). Each cell contributes a partial product toward BConv. Interleaved outputs flow directly to the subsequent MDC NTT, forming one macro-pipeline during ModChange (INTT $\rightarrow$ BConv $\rightarrow$ NTT).

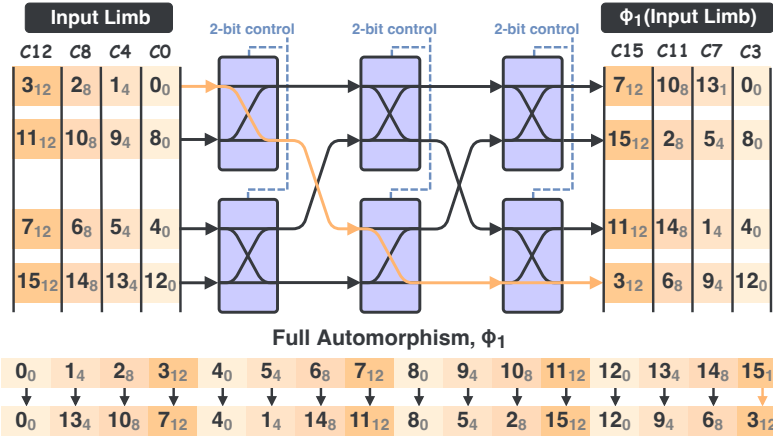


Fig. 7. Using a 4-to-4 Beneš network to perform the ϕ_1 automorphism on the first limb of the interleaved, natural-ordered polynomial introduced in Figure 4. Note that only *local* permutations occur between the inputs at each cycle.

4.4 Automorphisms

We now describe how Osiris efficiently performs automorphisms. The automorphism applies the index permutation $\phi_r : i \rightarrow i \cdot 5^r \bmod N$ to each limb of a polynomial in the evaluation representation and in natural order. This permutation cyclically shifts values in the underlying vector message by r slots to the left.

The authors of ARK [47] note that with ring degree $N = 2^{16}$, the automorphism can be decomposed into 2^8 smaller permutations of 2^8 indices. More generally, we can build any automorphism from permutations to subsets of input indices if each subset, S , satisfies two conditions: $|S|$ is a power of two, and the indices in S are evenly spaced by $\frac{N}{|S|}$. When $|S| = N$, we recover the original automorphism. This property is fortuitous for Osiris since the MDC unit produces $p = 512$ evenly spaced elements per cycle in natural order. Therefore, when an automorphism is required, the MDC's output can be directly passed to any p -to- p permutation network, completing the operation over $\frac{N}{p}$ cycles. In Osiris, we choose to perform these permutations with a 512-to-512 Beneš network.

The Beneš network belongs to a family of multi-stage interconnection networks designed to simplify all-to-all communication. A p -to- p Beneš network consists of $2 \log_2(p) - 1$ stages. Each stage contains $\frac{p}{2}$ simple 2-input switches that collectively build arbitrary permutations over multiple stages. Fixed connections link the stages together.

Figure 7 visualizes a smaller 4-to-4 Beneš network performing ϕ_1 on the first limb of an interleaved polynomial in natural order with $N = 16$ and $\ell = 3$. We highlight three features of Figure 7. First, since input limbs are interleaved, elements from the same limb enter the network once every $\ell + 1 = 4$ cycles. Second, subsets of $p = 4$ indices, evenly spaced by $\frac{N}{p} = 4$, enter the Beneš network per cycle. Since each subset meets the criteria above, any automorphism can be decomposed into local permutations of these subsets. Third, ϕ_1 permutes each element *from* index i *to* index $i \cdot 5^1 \bmod 16$. We highlight the path that maps index 3 to index $3 \cdot 5^1 \bmod 16 = 15$, alongside the full ϕ_1 automorphism.

4.5 Hadamard Unit and Key Generation

Osiris also includes a unit responsible for all point-wise multiplications and additions present in CKKS. This *Hadamard* unit consists of $p = 512$ 1D systolic arrays, each of height d_{num} , which help

to efficiently perform inner products with key-switching keys. Each cell contains four modular multipliers and adders to support both KeyMult and diagonal multiplication simultaneously, which prevents stalling in our proposed dataflow. We further employ PRNG key generation, proposed by CraterLake [65] and also adopted in SHARP [46], to generate half of all key-switching keys on-chip, significantly alleviating off-chip memory pressure.

5 Osiris and the GSC Dataflow

In this section, we show how the systolic units from Section 4 are combined to build the complete Osiris architecture. We begin with a discussion of on-the-fly limb generation. Then, we present our novel GSC dataflow for matrix-vector products and show how it can be efficiently mapped onto Osiris.

On-the-fly Limb Extension: We adopt the OF-Limb extension technique originally proposed for use in hardware acceleration by ARK [47] to improve arithmetic intensity. OF-Limb uses a single precomputed limb (q_0) of the plaintext diagonal (coefficient representation) to generate all other limbs. The optimization saves significant off-chip communication at the cost of additional computation to generate them on-chip. We can generate any i 'th limb from this q_0 -limb through the following equation:

$$[P]_{q_i} = \text{NTT}([P_{\text{coeff}}]_{q_0}) \bmod q_i$$

Therefore, OF-Limb is performed with Osiris's MDC units. We use this technique to generate *all* plaintext diagonals in the matrix-vector products in bootstrapping and in the linear and convolutional layers of neural networks.

GSC Dataflow: To maximize the arithmetic intensity of matrix-vector products that use the BSGS algorithm, we propose a novel GSC dataflow. Recall from Algorithm 3 that the BSGS algorithm reduces the number of ciphertext rotations in a matrix-vector product with n non-zero diagonals from n to $n_1 + n_2$, where $n_1 n_2 = n$. Broadly, the goal of the GSC dataflow is to mask the latency of loading each baby-step rotation key with diagonal generation using OF-Limb. We visualize this dataflow alongside a high-level overview of the double-hoisting algorithm in Figure 8 using the same matrix-vector product introduced in Section 3.

We highlight three features of Figure 8. First, rotations to the *same* input ciphertext share a common Decompose output. Therefore, we can perform this computation only once, *hoisting* it out and reusing its result for all n_1 baby-step rotations. Similarly, Bossuat et al. [9] show that the ModDown operation commutes with addition, e.g., $\sum_i \text{ModDown}(ct_i) = \text{ModDown}(\sum_i ct_i)$. Therefore, a similar *second* level of hoisting can be performed when summing all giant steps. We cache the Decompose output to leverage its significant reuse.

Second, we use OF-Limb to generate the limbs of *all* plaintext matrix diagonals. Since each baby-step rotation key contributes to n_2 partial products, we naturally group the diagonals we generate into n_1 sets of n_2 diagonals to minimize off-chip memory accesses. While our MDC units are generating the current set of n_2 matrix diagonals, we load the next baby-step rotation key and use our Hadamard and automorphism units to prepare the ciphertext's next baby-step rotation. Thus, if the latency of fetching a rotation key is less than the latency of generating n_2 plaintext diagonals, then DRAM accesses are completely masked by computation. We deliberately size Osiris's width with this step in mind.

Third, since we iteratively generate each set of n_2 partial products, we can accumulate these products to reserved on-chip SRAM locations, reducing the size of our working set. When the ideal number of giant steps exceeds what can be stored on-chip, we find it more efficient to simply reduce the number of giant steps in the BSGS algorithm rather than send partial products off-chip. Since each giant step rotates a *unique* ciphertext, we cannot hoist out a common Decompose step.

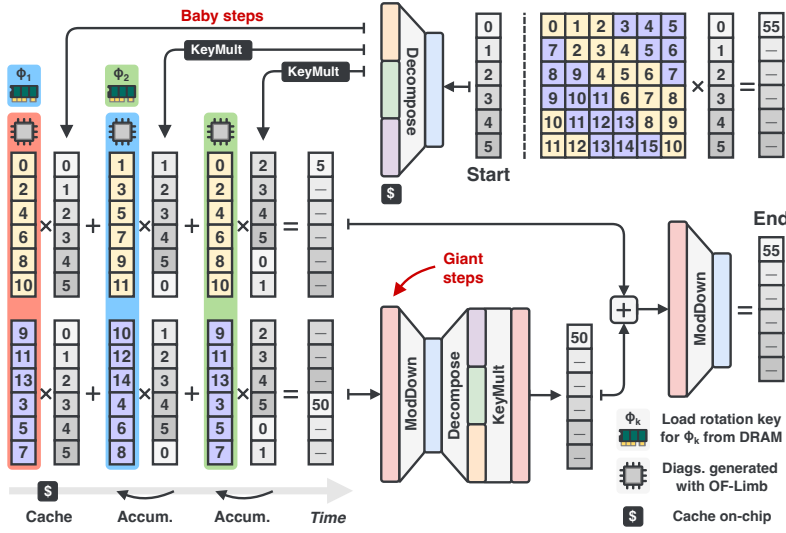


Fig. 8. Visualizing how the GSC dataflow masks rotation key DRAM accesses with OF-Limb generation using the BSGS example introduced in Figure 1(b).

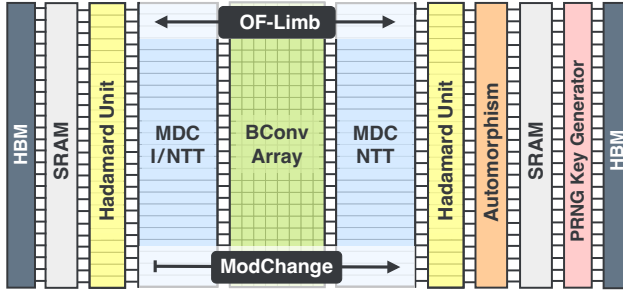


Fig. 9. The complete Osiris architecture. Arrows indicate the flow of data for each of the highlighted kernels.

Consequently, giant steps are more expensive than baby steps, and the optimal ratio of n_1/n_2 is large, often varying between 8 and 16 [9]. We set the maximum number of giant steps held on-chip to be $n_2 = 4$ in the highest multiplicative level matrix-vector products in bootstrapping.

Architecting Osiris: Figure 9 presents the full Osiris architecture. An important distinction between Osiris and prior FHE accelerators is that we dedicate *separate* units to each step of the ModChange (INTT $\rightarrow$ BConv $\rightarrow$ NTT) routine. This difference is motivated by our choice to feed limbs through Osiris in an *interleaved* fashion. Recall from Section 4 that interleaving resolves the differences between the desired input order of I/NTT and BConv and enables the units to operate together in lockstep. Therefore, when mapped onto Osiris, it is useful to view the ModChange operation as a single, macro-pipeline with a throughput of one output limb every $\frac{N}{p}$ cycles.

We also note that while the BSGS algorithm reduces the number of ciphertext rotations in matrix-vector products, it does not reduce the number of plaintext diagonal multiplications. When the number of non-zero diagonals is large, a significant fraction of end-to-end latency comes from OF-Limb. To mitigate this bottleneck, we leverage the bidirectional nature of pipelined NTT architectures to reverse the direction of our ModChange's MDC INTT unit for the duration

of OF-Limb. Since OF-Limb generates diagonals through repeated NTTs, this INTT unit would otherwise be left idle. This strategy doubles the throughput of OF-Limb, and we place Hadamard units on each side of Osiris to generate and accumulate partial products.

6 Methodology

Baselines: We evaluate our CPU baseline using Lattigo v5.2 on an h3-standard-88 GCP instance with an Intel Xeon Platinum 8481C processor clocked at 2.7 GHz. We take SHARP [46] as our accelerated baseline. Like SHARP, we assume a bit-width of 36 bits with double-prime scaling in bootstrapping. We also adopt a recent algorithmic optimization from MAD [2] to fuse the ModDown and Rescale stages within each HMult. Its impact is shown in Section 7.3.

Modeling: Given the predictability of performance in the systolic arrays used by Osiris, we developed an analytical performance model. First, we tested the systolic design of each kernel with functional simulation to verify correctness. We then outputted kernel traces from Lattigo running neural inferences to appropriately model the order of operations. Kernels are mapped to corresponding units and timings for each used to estimate total runtime.

Our goal in estimating power and area is to be commensurate with prior work. To do this, we normalize estimates to 7 nm. First, we use the Montgomery multiplier and modular addition units from BTS [50], with area and power reported as $3833 \mu\text{m}^2$, $325 \mu\text{m}^2$, and 1.35 mW, 0.08 mW, respectively. The remaining components were synthesized using a commercial 28nm library available to us and scaled to 7 nm using scaling factors of 3.6 for area and 3.3 for power, as assumed in prior work [55, 57, 73]. We model power using Synopsys Design Compiler Version T-2022.03-SP2 [72] and assume the worst-case switching to conservatively model the distribution of data in cryptographic workloads. Estimates of other logic (the crossbar, registers, MUXs, etc) were synthesized each using the library and scaling down as reported above. We further generated SRAMs using a commercial SRAM compiler and estimate the area and power in 7 nm using the same scaling factors. Our average SRAM density is $0.49 \text{ mm}^2/\text{MB}$, which is similar to SHARP's. We have a total SRAM size of 256 MB for keys, partials, and decomposed polynomials; an additional 32 MB of space is needed for MDC buffering. Finally, we assume the same wire overhead assumption as in ARK [47], adding an additional 10% area to the total design. We clock the chip at 1 GHz.

Benchmarks: We evaluate Osiris on a range of convolutional and fully-connected neural networks seen in prior work [23, 24, 46, 47, 50, 52, 56, 65]. Our benchmarks are as follows: **MLP** is a three-layer fully-connected network from SecureML [56] for MNIST. **LoLA** is a three-layer convolutional neural network from LoLA-CryptoNets [12] also targeting MNIST. It has been previously evaluated in CraterLake [65]. **LeNet** is a four-layer convolutional neural network adopted from Orion [26] and the CHET and EVA FHE compilers [23, 24]. It is the largest of their LeNet benchmarks and consists of two strided convolutions and two fully connected layers. **Bootstrap:** We evaluate the state-of-the-art fully-packed bootstrapping algorithm from Bossuat et al. [9]. **ResNet-20** : We re-implement prior work [52], replacing convolutions with their analogous matrix-vector products, leaving all else unchanged, including the use of high-degree composite polynomial approximations to ReLU. **HELR-1024** [36] trains an encrypted binary classifier on a subset of MNIST images of size 14×14 . Each epoch performs a forward and backward pass on one mini-batch of size 1024. Latency is reported as the average runtime per epoch. **Sorting** performs bitonic sorting [38] on 2^{14} elements in the range (0, 1). Each comparison is made by compositing twelve 7-degree polynomials. We note that sparsely-packed bootstrapping (when fewer than $\frac{N}{2}$ slots are filled with data) is performed in sorting, HELR-1024, and ResNet-20, which is consistent with prior work.

We also adopt the amortized mult time per slot metric ($T_{\text{mult, a/s.}}$) introduced in [43] and propose a similar metric that we denote as *amortized matrix-vector mult time per slot* defined in Equation (1),

Table 2. Residual Parameter Sets ($\ell \leq L_{\text{eff}}$) Used to Evaluate Osiris

Set	N	n	L_{eff}	α	dnum	q_0	Δ/q_i	p_i	h
I	2^{14}	2^{14}	5	2	3	36	30	36	192
II	2^{14}	2^{14}	7	4	2	36	30	36	192
III [†]	2^{16}	2^{15}	8	5	2	48	36	36	2048
IV [†]	2^{16}	2^{15}	9	5	2	48	36	36	2048

Moduli (q_i , p_i) and scales (Δ) are in bits. h denotes the Hamming weight of each parameter set's secret key.

[†]We apply double-prime scaling to the default bootstrapping parameters in Lattigo v5.2 [1], and separately set $\alpha = 14$. We further set $h = 2048$ for 128-bit security as $\log_2 QP_{\text{max}} = 1630$; q_0 is split into two primes.

for a matrix with d non-zero diagonals. This metric is conceptually similar to $T_{\text{mult}, \text{a/s.}}$, however aligns more closely with order of operations found in neural networks.

$$T_{\text{M} \times \text{v} : \#d, \text{a/slot}} = \frac{T_{\text{boot}} + \sum_{\ell=0}^{L-L_{\text{boot}}} T_{\text{M} \times \text{v} : \#d(\ell)}}{L - L_{\text{boot}}} \cdot \frac{2}{N}. \quad (1)$$

7 Evaluation

In this section, we evaluate Osiris on the aforementioned benchmarks and discuss the benefits afforded by systolic architectures in FHE. We begin by quantifying the implications of hoisting in hardware acceleration. Then, we compare Osiris against prior work. We close with sensitivity studies that demonstrate how Osiris's balanced design enables near-linear scaling trends with compute and bandwidth.

7.1 Benchmark Analysis

Table 3 reports the latency of each benchmark using the parameter sets from Table 2 on Osiris and with **non-hoisted** (NH), **single-hoisted** (SH), and **double-hoisted** (DH) BSGS matrix $\times$ vector algorithms. Table 3 also presents Osiris's improvements over the corresponding CPU implementations.

We highlight two observations in Table 3. First, hoisting provides noticeable performance improvements despite lower arithmetic intensity. However, the benefits of hoisting are more pronounced at higher bandwidths. For example, double hoisting reduces the total modular multiplications in fully-packed bootstrapping on Osiris by 1.68 $\times$ compared to BSGS without hoisting. However, with 1 TB/s of HBM, we observe only a 1.09 $\times$ improvement in latency. This discrepancy arises because we are unable to mask the latency of loading baby-step rotation keys on-chip with the computations of OF-Limb extension in double hoisting. As such, the design stalls for 13.9% of its total execution time. Conversely, BSGS without hoisting is mostly compute-limited; significantly less performance gains are achieved at higher bandwidths.

At 2 TB/s, the stalls in double hoisting are reduced to just 0.05 ms, leading to a 1.47 $\times$ improvement over BSGS without hoisting and approaching the theoretical limit. This is promising given the advent of HBM3 [40] and NVIDIA H100 GPUs [59] now boasting an aggregate 8 TB/s of bandwidth. We report latencies at 1 TB/s to be commensurable with prior work [46, 47, 65], however we feel it is now important to consider bandwidths beyond 1 TB/s in FHE accelerators given the pace of bandwidth scaling. We further explore scaling bandwidth and compute in Section 7.3.

Second, we report speedup ranging from 4350 $\times$ to 20200 $\times$ at 1 TB/s, with benchmarks that require bootstrapping seeing the largest improvement. The higher multiplicative levels in bootstrapping entail significantly more work than operations at lower levels. Osiris (and systolic architectures in general) are well-suited for kernels with large, consistent streams of data. The matrix-vector

Table 3. Benchmark Latencies Using NH, SH, and DH BSGS

Benchmark	Set	NH	SH	DH	CPU	Im-prov.
$T_{\text{mult, a/s.}} (ns)$	III	10.5	9.89	9.58	68 μs	$\times 6433$
$T_{\text{Mv: 128, a/s.}} (ns)$	III	18.9	17.3	17.4	112 μs	$\times 6438$
MLP (μs)	I	124	125	130	0.57s	$\times 4587$
LoLA (μs)	I	95.5	96.7	97.9	0.47s	$\times 4931$
LeNet (μs)	II	676	663	780	2.94s	$\times 4349$
Bootstrap (ms)	III	3.00	2.82	2.70	17.3s	$\times 6417$
HELR-1024 (ms)	III	2.67	2.61	2.46	15.9s	$\times 6290$
ResNet-20 (ms)	IV	102	97.6	93.6	620s	$\times 7189$
Sorting (s)	IV	1.26	1.20	1.14	23.0K	$\times 20.2K$
(a) All benchmarks evaluated at 1 TB/s, $p = 512$.						
Bootstrap (ms)	3.04	2.74	2.81	1.92	17.3s	$\times 9010$
HELR-1024 (ms)	2.71	2.49	2.52	2.13	15.9s	$\times 7465$
ResNet-20 (ms)	102	93.6	95.9	69.4	620s	$\times 8934$
Sorting (s)	1.28	1.16	1.22	0.85	23.0K	$\times 27.1K$

(b) Intensive benchmarks also evaluated at 2 TB/s, $p = 512$.

products present in bootstrapping embody this as they contain many sequential BConv and I/NTT operations. At the same time, the increased arithmetic intensity provided by OF-Limb extension is greatest when ℓ is large, as more plaintext diagonals must be generated. Therefore, while Osiris performs well across a spectrum of workloads, it excels in the most computationally-intense scenarios. The growing complexity of FHE applications is a promising trend for Osiris.

7.2 Comparisons with Prior Work

Latency: We now compare Osiris’s performance against the prior work of BTS [50], CraterLake [65], ARK [47], and SHARP [46]. Figure 10 focuses on the latency of bootstrapping and ResNet-20 inference, since these two benchmarks are common across all designs. All compared designs target 128-bit security with $N = 2^{16}$, with the exception of BTS which uses $N = 2^{17}$. Where parameter sets differ (e.g., in L or α), we normalize by L_{eff} to ensure fair comparison.

We highlight two features of Figure 10. First, Osiris achieves state-of-the-art on both benchmarks at a bandwidth of 1 TB/s. We also expect to see better performance scaling with bandwidth than SHARP [46], where an alternative **minimum key-switching (Min-KS)** strategy [47] (incompatible with hoisting) is used to boost arithmetic intensity. For instance, ARK [47], using the same technique, reports only a 1.07 $\times$ improvement on ResNet-20 at 2 TB/s, whereas Osiris improves by 1.35 $\times$.

Second, we normalize latencies relative to BTS [50], which, to the best of our knowledge, is the only compared design that does not adopt the BSGS algorithm. To fairly compare with BTS, we also implement a bootstrapping variant that excludes both BSGS and hoisting. At 1 TB/s, this variant achieves the highest multiplier utilization of 76.4% and a bootstrap latency of 5.64 ms, a 4.05 $\times$ improvement over BTS when normalized by L_{eff} . This demonstrates Osiris’s flexibility in supporting a wide variety of workloads without hoisting.

Power/area: Table 4 compares the hardware in Osiris against prior work. Similar to SHARP [46], we use optimistic 14 nm to 7 nm area (2.8 $\times$) and power (2.2 $\times$) scaling factors from [58] to fairly compare

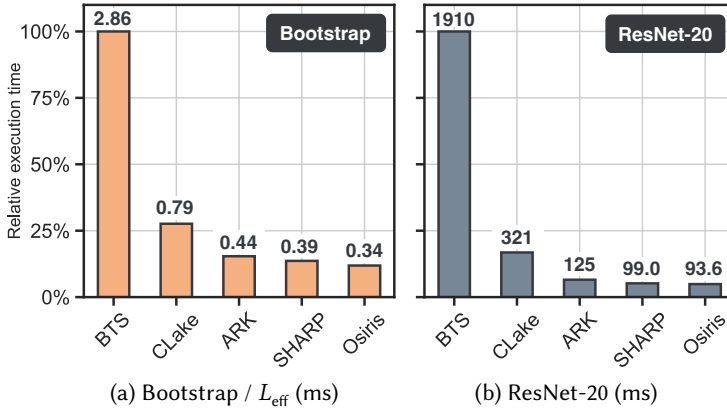


Fig. 10. Comparison of Osiris's bootstrap (normalized by L_{eff}) and ResNet-20 performance versus prior work. In Osiris, we set $L_{\text{eff}} = 8$, similar to ARK [47] and SHARP [46]. True latency values are shown above each bar.

Table 4. Comparing Osiris's Hardware Against Prior Work

ASIC Design	BW (TB/s)	Bit Width	SRAM (MB)	Area (mm ²)	Power (W)
BTS	1	64	512	373.6	163.2
ARK	1	64	588	418.3	281.3
SHARP	1	36	198	178.8	94.70
CLake [†]	1	28	256	222.7	169.7
Osiris	1	36	210	211.6	123.8

Avg. power reported for ResNet-20 following SHARP [46].

[†]Scaled from 14/12 nm to 7 nm to match Osiris and [46, 47, 50].

Table 5. Component-wise Area (mm²) and Power (W) for ResNet-20

	SRAM	HBM	I/NTT	BConv	Hada.	Auto.	WO.
Area	83.0	29.6	54.4	12.9	8.91	0.56	19.2
Power	19.4	31.8	39.8	15.1	5.61	0.80	11.3

MDC buffers are included in I/NTT area/power. Wiring overhead is denoted as WO.

against CraterLake [65]. Notably, since modular multiplier area and power scales superlinearly with bit width [46], CraterLake and SHARP can achieve a higher total compute capability for a similar area and power budget. A further component-wise area and power breakdown for ResNet-20 is shown in Table 5. Since we allocate *two* MDC I/NTT units in Osiris, the contribution of these units to total area is 2.63× higher than SHARP. In contrast, our BConv hardware consumes just 6.1% (12.2%) of area (power).

Hoisting vs. Min-KS: ARK [47] introduced a Min-KS technique to increase the arithmetic intensity of bootstrapping, and this approach was later adopted in SHARP. This method reduces the number of unique rotation keys required for bootstrapping by leveraging the structure of its matrix-vector products. Specifically, rotation amounts exhibit an arithmetic progression (e.g., r , $2r$, $3r$) that can be iteratively generated through rotations by r .

We find this technique has two drawbacks. First, each rotation is applied to a unique ciphertext and therefore hoisting cannot be utilized. As such, Osiris requires 2.85× fewer modular multiplications

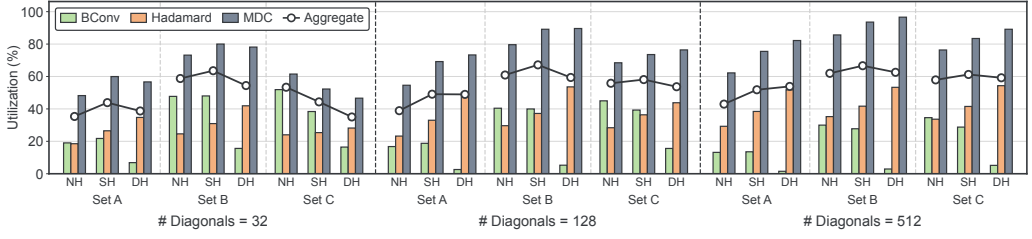


Fig. 11. Per-component hardware utilization for BSGS matrix-vector products on Osiris. Swept across parameter sets (Table 6), number of non-zero diagonals (32–512), and hoisting strategies (NH: non-hoisted, SH: single-hoisted, DH: double-hoisted).

Table 6. Parameter Sets Used in Matrix-vector Product Sweep in Figure 11

Set	N	L	α	dnum	q_0	Δ/q_i	p_i	$\log_2 QP$
A	2^{14}	6	8	1	48	36	36	552
B	2^{16}	14	16	1	48	36	36	1128
C	2^{16}	30	16	2	48	36	36	1704

Moduli (q_i , p_i) and scales (Δ) are in bits. $\log_2 QP \leq 1792$ is 128-bit secure when $N = 2^{16}$.

than SHARP across all matrix-vector products in bootstrapping. Second, it shifts SHARP into a compute-limited regime, with its HBM utilization dropping to just 23% during bootstrapping. As a result, significant performance improvements will only come from scaling compute, not bandwidth. In contrast, Osiris, and hoisting, scales with both. For example, our ResNet-20 latency decreases from 93.6 ms to 69.4 ms when only doubling bandwidth, and to 47.3 ms when doubling both bandwidth and compute.

We note that hoisting fundamentally increases bandwidth requirements. The size of our architecture must be carefully chosen to ensure we remain balanced. Without OF-Limb [47] it is likely that hoisting would not be a viable technique on hardware accelerators. Moreover, we require two separate mapping strategies: one for all matrix-vector products, and one for all NH operations, such as the modular reduction within bootstrapping and the composite approximations to ReLU [52].

7.3 Sensitivity Study

Per-component utilization: Figure 11 compares per-unit utilization across a range of matrix-vector products and CKKS parameters, with each configuration subdivided into NH, SH, and DH BSGS algorithms. Table 6 lists the chosen parameter sets, including one with ring degree 2^{14} to analyze Osiris’s support for smaller workloads without bootstrapping.

Across all configurations, our MDC units see the highest utilization, and this utilization generally increases with increasing matrix-vector product complexity. This effect occurs because OF-Limb utilizes *both* MDC units simultaneously. When transitioning from diagonal generation to ModChange operations, the full MDC pipeline must drain before ModChange can begin. Linear transforms with fewer diagonals incur this switching penalty more frequently, as they have fewer giant steps. Additionally, the fraction of modular multiplications performing OF-Limb grows with the number of non-zero diagonals; for example, in Set C, this fraction increases from 61.1% to 90.8% when scaling from 32 to 512 diagonals and using double-hoisting.

We also find that BConv utilization decreases with increased hoisting and remains relatively low compared to other components. This reflects the intended behavior: hoisting predominantly

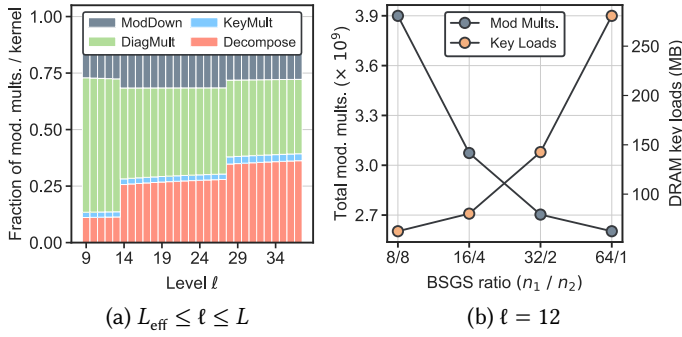


Fig. 12. Exploring the impact of level, ℓ , and BSGS ratio on the performance of a DH matrix-vector product with 64 non-zero diagonals. DiagMult includes OF-Limb.

eliminates expensive ModChange routines, directly reducing BConv's required workload. In Set C with 512 diagonals, BConv accounts for 17.8% of multiplications without hoisting versus 2.6% with double hoisting.

Finally, despite Osiris targeting workloads with bootstrapping, the smaller Set A still maintains high utilization between 35.3% and 53.9%. With ring degree $N = 2^{14}$, all I/NTTs skip the first two stages of each MDC (designed for $N \leq 2^{16}$), leaving their multipliers unused. In principle, one can and should clock gate these multipliers to reduce power consumption when operating at smaller ring degrees.

BSGS exploration: In Figure 12, we highlight the tradeoffs that occur when evaluating matrix-vector products using the double-hoisting BSGS algorithm [9] on Osiris. In Figure 12(a), we split the algorithm into its core kernels and evaluate a matrix-vector product with 64 non-zero diagonals (similar to the matrices found in bootstrapping) for levels $L_{\text{eff}} \leq \ell \leq L$. In Figure 12(b), we focus on just a single matrix-vector product at $\ell = 12$, varying the BSGS ratio (n_1/n_2) from 1 to 64 and plotting the number of modular multiplications alongside rotation key loads from DRAM. Recall from Section 5 that a goal of the GSC dataflow is to mask the latency of loading each baby-step rotation key by generating n_2 matrix diagonals with OF-Limb. When the number of giant steps, n_2 , is sufficiently large, enough diagonals can be generated to ensure that all loads are masked. For instance, in Figure 12(b), a BSGS ratio of 1 avoids stalls entirely and maintains high multiplier utilization. However, when moving beyond $n_2 = 4$ to $n_2 = 1$, the design stalls for 35% of its execution time, reducing utilization to just 15%. This increases our latency on Osiris by $1.96\times$ despite $1.50\times$ fewer modular multiplications. Thus, while key multiplication contributes only marginally to modular operations (Figure 12(a)), it is often the deciding factor for the optimal BSGS ratio on Osiris.

Dataflow optimizations: In Figure 13, we isolate the impacts of accelerator-specific algorithmic optimizations and the effects of scaling compute and bandwidth on ResNet-20 performance. In Figure 13(a), we begin with a baseline implementation of Osiris at 1 TB/s with neither the GSC dataflow nor limb interleaving and build to a design that incorporates both alongside ModDown fusing. By adopting the GSC dataflow for ResNet-20, we reduce stalls from loading rotation keys by 113 ms (56 ms). Limb interleaving further boosts PE utilization from 8% (14%) by preventing additional stalls between our I/NTT and BConv hardware in each ModChange operation. ModDown fusing further reduces ResNet-20 runtime by 7% (8%).

We also note that ResNet-20 latency roughly halves as we double both memory bandwidth and compute. The near linear scaling trends seen here are deliberate and follow naturally from scaling

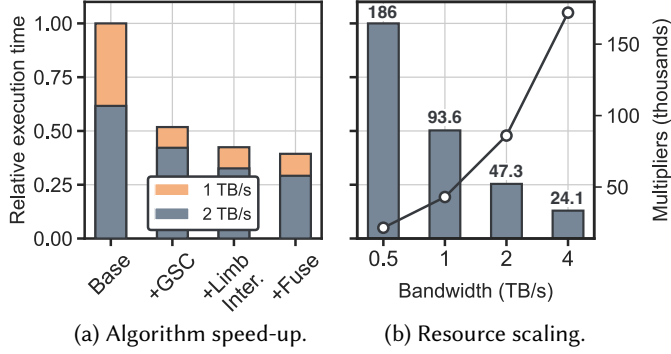


Fig. 13. Quantifying the impact of algorithmic optimizations and bandwidth scaling on Osiris's ResNet-20 latency.

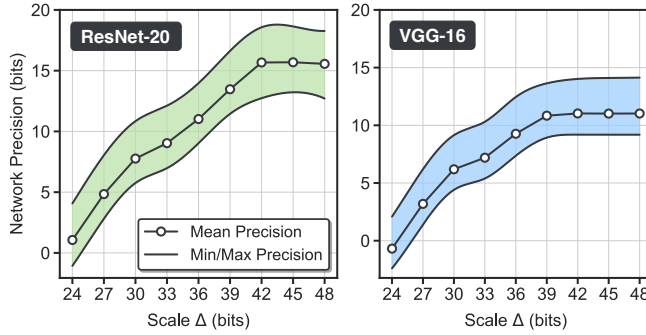


Fig. 14. Analyzing the impact of input scaling factor on inference precision for ResNet-20 and VGG-16. Precision plateaus at scales above $\Delta = 2^{40}$, but degrades noticeably below $\Delta = 2^{30}$, affecting network accuracy.

Osiris's array width. This highlights how the simple, lockstep execution a systolic architecture can target a variety of applications and environments. To support array widths greater than $p = 512$, we suggest adopting a cluster-based approach similar to ARK [47] and SHARP [46] to mitigate the increase in wiring overhead between MDC stages and within the Beneš network.

Bit width vs. precision: In Figure 14, we use Orion [26] to evaluate how scaling factors ranging from 24 to 48 bits affect inference precision in ResNet-20 and VGG-16 (270K and 14.7M parameters, respectively). Precision is measured as $-\log_2(\epsilon)$, where ϵ is the L1 prediction error relative to PyTorch. Metrics are averaged over 20 inferences per scaling factor. We find that in both networks, input scaling factors can be reduced to 2^{30} before precision loss begins to affect accuracy. This is lower than the 2^{36} required in SHARP [46], which we attribute to Orion's *errorless* rescaling policy. In the canonical client-server FHE setup, the bit width of each small modulus, $\log_2 q_i$, is independent of security. As long as the aggregate bit width $\log_2 QP$ remains below the security threshold, 128-bit security is maintained.

Rational rescaling: Smaller bit widths (and thus modular multipliers) can be achieved using recent techniques such as grafting [13] and rational rescaling [45, 66], which decouple scale factors from the q_i modulus chain. For instance, Cheddar [45] targets 32-bit GPU arithmetic by using 25-30 bit moduli, where rescaling both removes and adds primes to maintain scale factors up to 2^{40} . This increases L_{eff} , with the tradeoff that rescale operations become more general ModChange routines in non-bootstrapping operations. We refer readers to Section 3 of Cheddar for further details.

We implement rational rescaling [45] on Osiris during the matrix-vector products in Figure 11 and observe modest latency overheads between $1.01\times$ and $1.03\times$. In return, the technique enables 32-bit, rather than 36-bit, modular multipliers, reducing Osiris's chip area and power by 4.4% and 3.7%, respectively. This favorable tradeoff makes rational rescaling, like OF-Limb extension [47], a worthwhile optimization for FHE accelerators going forward.

8 Related Work

CPU/GPU: Multiple FHE software libraries are now available [1, 6, 15, 34, 67], providing high-performance CPU implementations. The high computational and memory bandwidth of GPUs provide a promising alternative to CPUs for running FHE. Early GPU efforts such as cuHE [22] and kernel fusion techniques [43] paved the way for subsequent optimizations. More recent work has achieved substantial speedups through memory-centric optimizations [43], fine-tuned NTT kernels [60], and Tensor Core utilization for modular arithmetic [27, 29]. In particular, 32-bit RNS implementations [45] exploit the native GPU integer datapath, while microarchitectural extensions such as GME [68] propose custom modular reduction units and locality-aware scheduling. Open-source libraries like FIDESlib [4] further lower the barrier to GPU-accelerated FHE by providing optimized CKKS kernels interoperable with existing frameworks. These advances have narrowed the gap between GPUs and custom ASICs, establishing strong software baselines that motivate continued algorithmic innovation in hardware designs like Osiris.

FPGAs: Prior work [3, 64, 75] employs FPGAs as lower-power, more customizable alternatives to GPUs for FHE acceleration. HEAX [64] targets ciphertext-ciphertext multiplication and reports two orders of magnitude speedup over CPUs, while FAB [3] is the first to support CKKS bootstrapping on FPGAs using double-hoisting optimizations [9]. However, while FPGA-based approaches are significantly more flexible than ASIC designs, they still lack the available on-chip memory to scale compute without bandwidth concerns, motivating the need for ASIC acceleration.

ASIC Acceleration: F1 [30] and Cheetah [63] were among the first ASIC designs to target homomorphic encryption. Since then, many works such as CraterLake [65], BTS [50], ARK [47], and SHARP [46] have targeted FHE, explicitly supporting bootstrapping. CraterLake [65] was the first to realize sub-second ResNet-20 latencies, and optimizations such as Min-KS and OF-Limb-extension from ARK [47] and SHARP [46] have further reduced latencies by $3\times$. Moreover, the use of 36-bit moduli in SHARP [46], coupled with PRNG key generation from CraterLake [65], have pushed FHE into the realm of practicality, tackling critical issues such as prohibitively large on-chip memories and low arithmetic intensity. More recently, unified accelerators such as Trinity [25] target multi-scheme support (CKKS, TFHE, and scheme conversion) within a single architecture through dynamic workload scheduling and hardware reuse.

Morphling [61] was the first to introduce systolic arrays for FHE acceleration within TFHE's bootstrapping operation. In particular, their 2D systolic array of vector PEs accelerates external products during blind rotations by accumulating directly in the transform-domain. Osiris applies systolic principles to CKKS, by instead directly accelerating matrix multiplication within BConv. While distinct operations, this convergence suggests that systolic architectures may be a natural fit for FHE acceleration, as different schemes [7, 33] expose similar opportunities for regular, predictable, and data-oblivious computation. Recent processing-in-memory approaches [48], algorithmic optimizations such as key decomposition switching [51], and streaming architectures [39, 62] further demonstrate the breadth of viable exploration in this space.

9 Conclusion

To the best of our knowledge, this is the first time CKKS has been mapped to a systolic architecture. The benefits of systolic architectures are many: simplified hardware, clear compute-memory

tradeoffs facilitating straightforward analytical analysis, and elimination of complex hardware structures (e.g., register files). The recent successes of systolic arrays in machine learning speak to the potential of the architecture. Through the development of the Osiris accelerator and giant-step centric dataflow, we have demonstrated a significantly simpler FHE accelerator compared to prior work while achieving state-of-the-art bootstrap performance.

Acknowledgments

We also thank Alhad Daftardar for the help in synthesizing our Beneš network. The views, opinions, and/or findings expressed are those of the authors and do not necessarily reflect the views of sponsors.

References

- [1] 2024. Lattigo v6. Retrieved from <https://github.com/tuneinsight/lattigo>
- [2] Rashmi Agrawal, Leo Castro, Chiraag Juvekar, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2023. MAD: Memory-aware design techniques for accelerating fully homomorphic encryption. In *Proceedings of the MICRO*.
- [3] Rashmi Agrawal, Leo de Castro, Guowei Yang, Chiraag Juvekar, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2022. FAB: An FPGA-based Accelerator for Bootstrappable Fully Homomorphic Encryption. arXiv:2207.11872. Retrieved from <https://arxiv.org/abs/2207.11872>
- [4] Carlos Agulló-Domingo, Óscar Vera-López, Seyda Guzelhan, Lohit Daksha, Aymane El Jerari, Kaustubh Shivdikar, Rashmi Agrawal, David Kaeli, Ajay Joshi, and José L. Abellán. 2025. FIDESlib: A fully-fledged open-source FHE library for efficient CKKS on GPUs. In *Proceedings of the 2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 1–3. DOI: <https://doi.org/10.1109/ISPASS64960.2025.00045>
- [5] Ehud Aharoni, Allon Adir, Moran Baruch, Nir Drucker, Gilad Ezov, Ariel Farkash, Lev Greenberg, Ramy Masalha, Guy Moshkovich, Dov Murik, Hayim Shaul, and Omri Soceanu. 2023. HeLayers: A tile tensors framework for large neural networks on encrypted data. *Proceedings on Privacy Enhancing Technologies* 2023, 1 (2023), 325–342. DOI: <https://doi.org/10.56553/popets-2023-0020>
- [6] Ahmad Al Badawi, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Ian Quah, Yuriy Polyakov, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Sponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. 2022. *OpenFHE: Open-Source Fully Homomorphic Encryption Library*. Cryptology ePrint Archive, Paper 2022/915. Retrieved from <https://eprint.iacr.org/2022/915>
- [7] Youngjin Bae, Jung Hee Cheon, Guillaume Hanrot, Jai Hyun Park, and Damien Stehlé. 2025. Fast Homomorphic Linear Algebra with BLAS. arXiv:2503.16080. Retrieved from <https://arxiv.org/abs/2503.16080>
- [8] Fabrice Benhamouda, Tancrede Lepoint, Claire Mathieu, and Hang Zhou. 2016. Optimization of Bootstrapping in Circuits. arXiv:1608.04535. Retrieved from <https://arxiv.org/abs/1608.04535>
- [9] Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2020. *Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-Sparse Keys*. Cryptology ePrint Archive, Paper 2020/1203. Retrieved from <https://eprint.iacr.org/2020/1203>
- [10] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory* 6, 3 (2014), 1–36.
- [11] Zvika Brakerski and Vinod Vaikuntanathan. 2011. Fully homomorphic encryption from ring-LWE and security for key dependent messages. In *Proceedings of the Annual Cryptology Conference*. Springer, 505–524.
- [12] Alon Brutzkus, Oren Elisha, and Ran Gilad-Bachrach. 2019. Low latency privacy preserving inference. In *Proceedings of the International Conference on Machine Learning*.
- [13] Jung Hee Cheon, Hyeonmin Choe, Minsik Kang, Jaehyung Kim, Seonghak Kim, Johannes Mono, and Taeyeon Noh. 2024. *Grafting: Decoupled Scale Factors and Modulus in RNS-CKKS*. Cryptology ePrint Archive, Paper 2024/1014. Retrieved from <https://eprint.iacr.org/2024/1014>
- [14] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2019. A full RNS variant of approximate homomorphic encryption. In *Proceedings of the 25th International Conference on Selected Areas in Cryptography–SAC 2018*. Springer, 347–368.
- [15] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 409–437.
- [16] Seonyoung Cheon, Yongwoo Lee, Dongkwan Kim, Ju Min Lee, Sunchul Jung, Taekyung Kim, Dongyoon Lee, and Hanjun Kim. 2024. DaCapo: Automatic bootstrapping management for efficient fully homomorphic encryption. In

- Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 6993–7010. Retrieved from <https://www.usenix.org/conference/usenixsecurity24/presentation/cheon>
- [17] James W. Cooley and John W. Tukey. 1965. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation* 19, 90 (1965), 297–301. Retrieved from <http://www.jstor.org/stable/2003354>
 - [18] Meghan Cowan, Deeksha Dangwal, Armin Alaghi, Caroline Trippel, Vincent T. Lee, and Brandon Reagen. 2021. Porcupine: A synthesizing compiler for vectorized homomorphic encryption. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 375–389. DOI : <https://doi.org/10.1145/3453483.3454050>
 - [19] Alhad Daftardar, Jianqiao Mo, Joey Ah-kiow, Benedikt Bünz, Ramesh Karri, Siddharth Garg, and Brandon Reagen. 2025. Need for zkSpeed: Accelerating hyperplonk for zero-knowledge proofs. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA’25)*. Association for Computing Machinery, New York, NY, USA, 1986–2001. DOI : <https://doi.org/10.1145/3695053.3731021>
 - [20] Alhad Daftardar, Jianqiao Mo, Joey Ah-kiow, Benedikt Bünz, Siddharth Garg, and Brandon Reagen. 2025. zkPHIRE: A Programmable Accelerator for ZKPs over High-degRee, Expressive Gates. arXiv:2508.16738. Retrieved from <https://arxiv.org/abs/2508.16738>
 - [21] Alhad Daftardar, Brandon Reagen, and Siddharth Garg. 2024. SZKP: A scalable accelerator architecture for zero-knowledge proofs. In *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques (PACT’24)*. Association for Computing Machinery, New York, NY, USA, 271–283. DOI : <https://doi.org/10.1145/3656019.3676898>
 - [22] Wei Dai. [n. d.]. cuHE: CUDA Homomorphic Encryption Library. Retrieved from <https://github.com/vernamlab/cuHE>
 - [23] Roshan Dathathri, Blagovesta Kostova, Olli Saarikivi, Wei Dai, Kim Laine, and Madan Musuvathi. 2020. EVA: An encrypted vector arithmetic language and compiler for efficient homomorphic computation. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM. DOI : <https://doi.org/10.1145/3385412.3386023>
 - [24] Roshan Dathathri, Olli Saarikivi, Hao Chen, Kim Laine, Kristin Lauter, Saeed Maleki, Madanlal Musuvathi, and Todd Mytkowicz. 2019. CHET: An optimizing compiler for fully-homomorphic neural-network inferencing. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2019)*. Association for Computing Machinery, New York, NY, USA, 142–156. DOI : <https://doi.org/10.1145/3314221.3314628>
 - [25] Xianglong Deng, Shengyu Fan, Zhicheng Hu, Zhuoyu Tian, Zihao Yang, Jiangrui Yu, Dingyuan Cao, Dan Meng, Rui Hou, Meng Li, Qian Lou, and Mingzhe Zhang. 2024. Trinity: A general purpose FHE accelerator. In *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 338–351. DOI : <https://doi.org/10.1109/MICRO61859.2024.00033>
 - [26] Austin Ebel, Karthik Garimella, and Brandon Reagen. 2025. Orion: A fully homomorphic encryption framework for deep learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS’25)*. Association for Computing Machinery, New York, NY, USA, 734–749. DOI : <https://doi.org/10.1145/3676641.3716008>
 - [27] Guang Fan, Mingzhe Zhang, Fangyu Zheng, Shengyu Fan, Tian Zhou, Xianglong Deng, Wenxu Tang, Liang Kong, Yixuan Song, and Shoumeng Yan. 2025. WarpDrive: GPU-based fully homomorphic encryption acceleration leveraging tensor and CUDA cores. In *Proceedings of the 2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, Las Vegas, NV, USA, 1187–1200.
 - [28] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive* (2012).
 - [29] Shengyu Fan, Zhiwei Wang, Weizhi Xu, Rui Hou, Dan Meng, and Mingzhe Zhang. 2023. TensorFHE: Achieving practical computation on encrypted data using GPGPU. In *Proceedings of the 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, Montreal, QC, Canada, 922–934. DOI : <https://doi.org/10.1109/HPCA56546.2023.10071017>
 - [30] Axel Feldmann, Nikola Samardzic, Aleksandar Krastev, Srini Devadas, Ron Dreslinski, Karim Eldefrawy, Nicholas Genise, Chris Peikert, and Daniel Sanchez. 2021. F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption (Extended Version). arXiv:2109.05371. Retrieved from <https://arxiv.org/abs/2109.05371>
 - [31] Franz Franchetti and Markus Püschel. 2011. FFT (Fast Fourier Transform). In *Proceedings of the Encyclopedia of Parallel Computing*. David Padua (Ed.), Springer, Boston, MA, 658–671. DOI : https://doi.org/10.1007/978-0-387-09766-4_243
 - [32] Mario Garrido. 2022. A survey on pipelined FFT hardware architectures. *Journal of Signal Processing Systems* 94, 11 (2022), 1345–1364.
 - [33] Craig Gentry and Yongwoo Lee. 2025. *Fully Homomorphic Encryption for Matrix Arithmetic*. Cryptology ePrint Archive, Paper 2025/1935. Retrieved from <https://eprint.iacr.org/2025/1935>
 - [34] Shai Halevi and Victor Shoup. 2014. *Algorithms in HElib*. Cryptology ePrint Archive, Paper 2014/106. Retrieved from <https://eprint.iacr.org/2014/106>

- [35] Shai Halevi and Victor Shoup. 2018. Faster homomorphic linear transformations in HELib. In *Proceedings of the Advances in Cryptology—CRYPTO 2018*. Hovav Shacham and Alexandra Boldyreva (Eds.), Springer International Publishing, Cham, 93–120.
- [36] Kyoohyung Han, Seungwan Hong, Jung Hee Cheon, and Daejun Park. 2018. *Efficient Logistic Regression on Large Encrypted Data*. Cryptology ePrint Archive, Paper 2018/662. Retrieved from <https://eprint.iacr.org/2018/662>
- [37] Kyoohyung Han and Dohyeong Ki. 2020. Better bootstrapping for approximate homomorphic encryption. In *Proceedings of the Topics in Cryptology—CT-RSA 2020 (Lecture Notes in Computer Science, Vol. 12006)*. Springer, Cham, 364–390. DOI : https://doi.org/10.1007/978-3-030-40186-3_16
- [38] Seungwan Hong, Seunghong Kim, Jiheon Choi, Younho Lee, and Jung Hee Cheon. 2021. Efficient sorting of homomorphic encrypted data with k-way sorting network. *IEEE Transactions on Information Forensics and Security* 16 (2021), 4389–4404. DOI : <https://doi.org/10.1109/TIFS.2021.3106167>
- [39] Yi Huang, Xinsheng Gong, Xiangyu Kong, Dibe Chen, Jianfeng Zhu, Wenping Zhu, Liangwei Li, Mingyu Gao, Shaojun Wei, Aoyang Zhang, and Leibo Liu. 2025. EFFACT: A highly efficient full-stack FHE acceleration platform. In *Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1143–1157. DOI : <https://doi.org/10.1109/HPCA61900.2025.00088>
- [40] JEDEC 2023. 2023. *High Bandwidth Memory DRAM (HBM3)*. Technical Report JESD238A. JEDEC Solid State Technology Association. Retrieved from <https://www.jedec.org/standards-documents/docs/jesd238a>
- [41] Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. arXiv:2304.01433. Retrieved from <https://arxiv.org/abs/2304.01433>
- [42] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA'17)*. Association for Computing Machinery, New York, NY, USA, 1–12. DOI : <https://doi.org/10.1145/3079856.3080246>
- [43] Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Hee Cheon, and Younho Lee. 2021. Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with GPUs. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021, 4 (2021), 114–148.
- [44] Chirag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. 2018. GAZELLE: A low latency framework for secure neural network inference. In *Proceedings of the 27th USENIX Conference on Security Symposium (SEC'18)*. USENIX Association, USA, 1651–1668.
- [45] Jongmin Kim, Wonseok Choi, and Jung Ho Ahn. 2024. Cheddar: A Swift Fully Homomorphic Encryption Library for CUDA GPUs. arXiv:2407.13055. Retrieved from <https://arxiv.org/abs/2407.13055>
- [46] Jongmin Kim, Sangpyo Kim, Jaewan Choi, Jaiyoung Park, Donghwan Kim, and Jung Ho Ahn. 2023. SHARP: A short-word hierarchical accelerator for robust and practical fully homomorphic encryption. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA'23)*. Association for Computing Machinery, New York, NY, USA, Article 18, 15 pages. DOI : <https://doi.org/10.1145/3579371.3589053>
- [47] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, Minsoo Rhu, John Kim, and Jung Ho Ahn. 2022. ARK: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse. In *Proceedings of the 2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, Chicago, IL, USA, 1237–1254. DOI : <https://doi.org/10.1109/MICRO56248.2022.00086>
- [48] Jongmin Kim, Sungmin Yun, Hyesung Ji, Wonseok Choi, Sangpyo Kim, and Jung Ho Ahn. 2025. Anaheim: Architecture and algorithms for processing fully homomorphic encryption in memory. In *Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1158–1173. DOI : <https://doi.org/10.1109/HPCA61900.2025.00089>
- [49] Sangpyo Kim, Wonkyung Jung, Jaiyoung Park, and Jung Ho Ahn. 2020. Accelerating number theoretic transformations for bootstrappable homomorphic encryption on GPUs. In *Proceedings of the 2020 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, Virtual Event, 264–275. DOI : <https://doi.org/10.1109/IISWC50251.2020.00033>
- [50] Sangpyo Kim, Jongmin Kim, Michael Jaemin Kim, Wonkyung Jung, John Kim, Minsoo Rhu, and Jung Ho Ahn. 2022. BTS: An accelerator for bootstrappable fully homomorphic encryption. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA'22)*. ACM, New York, NY, USA, 711–725. DOI : <https://doi.org/10.1145/3470496.3527415>
- [51] Liang Kong, Shengyu Fan, Xianglong Deng, Lei Chen, Guang Fan, Guiming Shi, Yilan Zhu, Geng Yang, Shoumeng Yan, and Mingzhe Zhang. 2025. HAWK: Fully homomorphic encryption accelerator with fixed-word key decomposition switching. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1719–1734.
- [52] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. 2022. Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In *Proceedings of the 39th International Conference on Machine Learning*. Kamalika Chaudhuri, Stefanie

- Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (Eds.), *Proceedings of Machine Learning Research*, Vol. 162, PMLR, 12403–12422. Retrieved from <https://proceedings.mlr.press/v162/lee22e.html>
- [53] Thomas Lenart and Viktor Owall. 2006. Architectures for dynamic data scaling in 2/4/8K pipeline FFT cores. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 14, 11 (2006), 1286–1290. DOI: <https://doi.org/10.1109/TVLSI.2006.886407>
- [54] Raghav Malik, Kabir Sheth, and Milind Kulkarni. 2023. Coyote: A compiler for vectorizing encrypted arithmetic circuits. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 118–133. DOI: <https://doi.org/10.1145/3582016.3582057>
- [55] Jianqiao Mo, Jayanth Gopinath, and Brandon Reagen. 2023. HAAC: A hardware-software co-design to accelerate garbled circuits. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA'23)*. ACM. DOI: <https://doi.org/10.1145/3579371.3589045>
- [56] Payman Mohassel and Yupeng Zhang. 2017. *SecureML: A System for Scalable Privacy-Preserving Machine Learning*. Cryptology ePrint Archive, Paper 2017/396. Retrieved from <https://eprint.iacr.org/2017/396>
- [57] Nanotechnology Products Database. 2023. TSMC 16FF+ (FinFET plus). (2023). Retrieved from [https://product.statnano.com/product/6775/16ff-\(finfet-plus\)](https://product.statnano.com/product/6775/16ff-(finfet-plus))
- [58] S. Narasimha, B. Jagannathan, A. Ogino, et al. 2017. A 7nm CMOS technology platform for mobile and high performance compute application. In *Proceedings of the 2017 IEEE International Electron Devices Meeting (IEDM)*. 29.5.1–29.5.4. DOI: <https://doi.org/10.1109/IEDM.2017.8268476>
- [59] NVIDIA. 2023. NVIDIA H100 Tensor Core GPU. Available: NVIDIA. Retrieved from <https://www.nvidia.com/en-us/data-center/h100/>
- [60] Ozgun Ozerk, Can Elgezen, Ahmet Can Mert, Erdinc Ozturk, and Erkay Savas. 2021. *Efficient Number Theoretic Transform Implementation on GPU for Homomorphic Encryption*. Cryptology ePrint Archive, Paper 2021/124. Retrieved from <https://eprint.iacr.org/2021/124>
- [61] Prasetyo, Adiwena Putra, and Joo-Young Kim. 2024. Morphling: A throughput-maximized TFHE-based accelerator using transform-domain reuse. In *Proceedings of the 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, Edinburgh, UK, 249–262.
- [62] Adiwena Putra, Prasetyo, Yi Chen, John Kim, and Joo-Young Kim. 2023. Strix: An end-to-end streaming architecture with two-level ciphertext batching for fully homomorphic encryption with programmable bootstrapping. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'23)*. Association for Computing Machinery, New York, NY, USA, 1319–1331. DOI: <https://doi.org/10.1145/3613424.3614264>
- [63] Brandon Reagen, Wooseok Choi, Yeongil Ko, Vincent Lee, Gu-Yeon Wei, Hsien-Hsin S. Lee, and David Brooks. 2020. Cheetah: Optimizing and Accelerating Homomorphic Encryption for Private Inference. arXiv:2006.00505. Retrieved from <https://arxiv.org/abs/2006.00505>
- [64] M. Sadegh Riazi, Kim Laine, Blake Pelton, and Wei Dai. 2020. HEAX: An architecture for computing on encrypted data. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*. Association for Computing Machinery, New York, NY, USA, 1295–1309. DOI: <https://doi.org/10.1145/3373376.3378523>
- [65] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Nathan Manohar, Nicholas Genise, Srinivas Devadas, Karim Eldefrawy, Chris Peikert, and Daniel Sanchez. 2022. CraterLake: A hardware accelerator for efficient unbounded computation on encrypted data. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA'22)*. Association for Computing Machinery, New York, NY, USA, 173–187. DOI: <https://doi.org/10.1145/3470496.3527393>
- [66] Nikola Samardzic and Daniel Sanchez. 2024. BitPacker: Enabling high arithmetic efficiency in fully homomorphic encryption accelerators. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS'24)*. Association for Computing Machinery, New York, NY, USA, 137–150. DOI: <https://doi.org/10.1145/3620665.3640397>
- [67] SEAL 2023. Microsoft SEAL (release 4.1). Retrieved from <https://github.com/Microsoft/SEAL>. Microsoft Research, Redmond, WA.
- [68] Kaustubh Shivdikar, Yuhui Bao, Rashmi Agrawal, Michael Shen, Gilbert Jonatan, Evelio Mora, Alexander Ingare, Neal Livesay, José L. Abellán, John Kim, Ajay Joshi, and David Kaeli. 2023. GME: GPU-based microarchitectural extensions to accelerate homomorphic encryption. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–15.
- [69] Sujoy Sinha Roy, Furkan Turan, Kimmo Jarvinen, Frederik Vercauteren, and Ingrid Verbauwhede. 2019. FPGA-based high-performance parallel architecture for homomorphic computing on encrypted data. In *Proceedings of the 2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 387–398. DOI: <https://doi.org/10.1109/HPCA.2019.00052>

- [70] N. P. Smart and F. Vercauteren. 2014. Fully homomorphic SIMD operations. *Designs, Codes, and Cryptography* 71, 1 (2014), 57–81. DOI : <https://doi.org/10.1007/s10623-012-9720-4>
- [71] Deepraj Soni, Negar Neda, Naifeng Zhang, Benedict Reynwar, Homer Gamil, Benjamin Heyman, Mohammed Nabeel, Ahmad Al Badawi, Yuriy Polyakov, Kellie Canida, Massoud Pedram, Michail Maniatakos, David Bruce Cousins, Franz Franchetti, Matthew French, Andrew G. Schmidt, and Brandon Reagen. 2023. RPU: The ring processing unit. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 272–282. DOI : <https://doi.org/10.1109/ISPASS57527.2023.00034>
- [72] Synopsys. 2022. Synopsys Synthesis Installation Notes Version T-2022.03 (March 2022). Retrieved from https://solvnet.synopsys.com/Install/installation_guide.jsp?id=212&releasedate=2022-03-07
- [73] TSMC. [n. d.]. 16/12nm technology. ([n. d.]). Retrieved from https://www.tsmc.com/english/dedicatedFoundry/technology/logic/l_16_12nm
- [74] Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi. 2023. HECO: Fully Homomorphic Encryption Compiler. arXiv:2202.01649. Retrieved from <https://arxiv.org/abs/2202.01649>
- [75] Yinghao Yang, Huaizhi Zhang, Shengyu Fan, Hang Lu, Mingzhe Zhang, and Xiaowei Li. 2023. Poseidon: Practical homomorphic encryption accelerator. In *Proceedings of the 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 870–881. DOI : <https://doi.org/10.1109/HPCA56546.2023.10070984>

Received 4 August 2025; revised 9 November 2025; accepted 22 December 2025